

1 The Read-Once Property and Token Matching

This note proves that the MUX-tree satisfies the token-matching invariants internally exactly when it is read-once.

Setup: the ROBDD-to-MUX-tree mapping

Let D be the ROBDD and let its MUX-tree be built by the Shannon expansion. Each nonterminal vertex v of D induces a *sub-ROBDD* D_v , the subgraph of all vertices reachable from v . D_v is itself an ROBDD with root v according to Bryant's paper.

The expansion step at v produces one MUX M_v . Its select input is the condition variable c_v , and its true and false data inputs are the outputs of the MUX-subtrees built from $D_{\text{high}(v)}$ and $D_{\text{low}(v)}$.

For a given firing of the producer block B_p , the *active path* is the unique root-to-terminal path of D . As established from the dominance of B_p (Section 5.1) and the higher-loop-level filtering (Section 5.3), a conditional block produces exactly one token for c_v per firing of B_p , and it does so precisely when v lies on the active path.

Two equivalent statements of read-once

A MUX-tree is *read-once* when every condition variable labels at most one MUX input. The following lemma restates this property in terms of the ROBDD structure, which is the form we use in the proof.

Lemma 1. *The MUX-tree generated by Shannon expansion is read-once if and only if, at every expansion step v , the two sub-ROBDDs $D_{\text{high}(v)}$ and $D_{\text{low}(v)}$ share no nonterminal vertex.*

Proof. Suppose first that some step v has $D_{\text{high}(v)}$ and $D_{\text{low}(v)}$ sharing a nonterminal vertex w . Then w occurs in the MUX-subtree built from $D_{\text{high}(v)}$ and also in the one built from $D_{\text{low}(v)}$. These two subtrees are the true and false data subtrees of M_v , so c_w labels at least an input in each of them. The MUX-tree then has at least two inputs labelled c_w , so it is not read-once.

Conversely, suppose c_w labels two distinct inputs. Let M_v be the MUX where the two paths (along the MUX-tree) to these inputs meet, so that the two inputs lie below different inputs of M_v . Neither can be the select input of M_v : the select is c_v , and by the ordered property c_v has a strictly smaller index than every nonterminal vertex below it, whereas both inputs carry the label c_w . So the two inputs lie in the true and false data subtrees of M_v , hence w occurs in both $D_{\text{high}(v)}$ and $D_{\text{low}(v)}$, and the step at v shares a nonterminal vertex. $\square$

Read-once implies token matching

Proposition 1. *If the MUX-tree is read-once, then on every firing of B_p it consumes each condition token exactly once and produces exactly one output token. The MUX-tree satisfies the token-matching invariants internally.*

Proof. By Lemma 1 the read-once property is equivalent to the two data subtrees at every step being vertex-disjoint. It means that every vertex of D is reached from the root by a unique path. In particular, every vertex of a sub-ROBDD D_v is reached only through v .

We prove by structural induction on the MUX-subtree T_v that, on a given firing:

- (i) if the active path passes through v , then T_v receives exactly the tokens of the vertices of D_v that lie on the active path, consumes each of them once, and produces exactly one output token. In particular, no token for an internal value is left unconsumed, and no required token for an internal value is missing;

- (ii) if the active path does not pass through v , then no vertex of D_v lies on the active path, T_v receives no token, and it produces no output token.

Statement (ii) is immediate from the unique-path property: every vertex of D_v is reached only through v , so if v is off the active path then no vertex of D_v is on it, and none of them sends a token into T_v . Next we prove (i).

Base case. When a data input of M_v is a constant or a single variable, (i) hold trivially.

Inductive step. (i)–(ii) hold for $T_{\text{high}(v)}$ and $T_{\text{low}(v)}$. When the active path passes through v . Then v produces one token. this token reaches only the select input of M_v . The active path leaves v along exactly one edge; assume it takes $\text{high}(v)$, the $\text{low}(v)$ case being symmetric. Then the active path does not pass through $\text{low}(v)$, so by (ii) the false subtree $T_{\text{low}(v)}$ receives no token and produces none, and the false data input of M_v carries no token. By (i) the true subtree $T_{\text{high}(v)}$ consumes exactly the active-path tokens inside $D_{\text{high}(v)}$, each once, and delivers one token to the true data input of M_v . Now M_v has a token on its select input and on its selected data input; it consumes both and emits one output token, and it does not consume the false data input, which carries no token. The tokens consumed inside T_v are the token of v together with those consumed by $T_{\text{high}(v)}$, which are exactly the vertices of D_v on the active path, each once, and the output is a single token. This is (i).

Applying the statement to the root of D , which lies on every active path. According to (i), the whole MUX-tree consumes exactly the tokens of the active-path vertices, each once, and produces a single output token. The MUX-tree is token-matched internally. $\square$

Why a non-read-once tree fails

If the MUX-tree is not read-once, then by Lemma 1 some step v has $D_{\text{high}(v)}$ and $D_{\text{low}(v)}$ sharing a nonterminal vertex w . So c_w labels at least one input in the true subtree of M_v and one in the false subtree. On a firing whose active path reaches w , the path passes through v and continues into only one of the two subtrees. By the reasoning of Proposition 1, the other subtree is not traversed and its copy of the token is never consumed. One copy of w 's token is thus produced but never consumed, violating the token-matching invariants.

2 Producers in a loop where the dominator is outside

Section 5.7 handles a γ -tree consumer by moving the start of the analysis from B_p to the dominator block B_d and building the decision graph from B_d . One situation there remains to be made precise. The producer B_p may lie in a loop that B_d is outside, so that B_p fires on every iteration of that loop while only the value of the final iteration is paired with the suppression expression on the main suppression BRANCH. The producer's token must then be filtered across nesting levels, in the same way that a condition token is filtered by demotion (Section 5.6). This section gives the construction.

Let C be the vertex set of the decision graph built from B_d , that is, the transitive closure of the control dependence of B'_c (Section 5.2). We split into two cases according to whether B_p is a vertex of this decision graph.

Case A: the producer is a decision vertex

If $B_p \in C$, its vertex lies on a cycle of the decision graph, because B_p sits in a loop deeper than B_d . The vertex therefore has a positive nesting level (Section 5.3). The producer's data token is demoted across levels exactly as the condition token of B_p is: it passes through the demotion branches of the leveled ROBDDs (Section 5.6). Nothing further is required.

Case B: the producer is not a decision vertex

If $B_p \notin C$, we deliver the producer in two steps.

1. *Suppression to the nearest decision vertex.* Construct a suppression circuit with B_p as producer and B_t as consumer, where B_t is the nearest vertex of C reached from B_p .
2. *Demotion through the nearest decision vertex.* The value suppressed by step 1 is then demoted using the demotion logic already built for B_t in the decision graph.

Existence and uniqueness of B_t . Walk forward from B_p in the local CFG and, on each path, record the first block of C reached along it (the false terminal being represented by a set of blocks). Suppose two paths record different blocks. Walk backward from B_p to the nearest preceding block $B \in C$ (such a block must exist), and let S be the successor of B through which B_p is reached, so that no block of C lies between S and B_p . From S one can then reach two different first blocks of C through B_p , so no single block post-dominates S . But Property 2 of Section 5.2 requires that some block post-dominate S , a contradiction. Hence every forward path from B_p records the same block B_t , and B_t post-dominates B_p .

B_t is the unique decision-graph vertex that post-dominates B_p . It is a genuine vertex, not the false terminal: otherwise B_p could not reach B_c , and its value would not appear on the γ -tree. Any other vertex B'_t that post-dominates B_p would lie after B_t on every path from B_p and would therefore post-dominate B_t ; but no decision-graph vertex post-dominates another, so no such B'_t exists.

B_t is not in a loop deeper than the producer. Suppose B_t lay in a loop L that does not contain B_p , with header h . If B_p is a definition, then since B_t post-dominates B_p , every path from B_p to B_t passes through h , where GSA places a μ -function. The definition from B_p is then replaced by the result of this μ , so it is not live beyond h and cannot be an input of the γ -tree at B_c , a contradiction. If instead B_p is a γ -predicate, then it is not post-dominated by any decision vertex other than the consumer, since otherwise it would not appear as a condition input of the γ -tree; hence B_t is the consumer block, which lies on the γ -tree and needs no deeper-loop treatment (proof in the thesis, will be improved). In either case B_t lies in no loop that excludes B_p .

The producer dominates the conditions of step 1. If B_p is a definition, we claim it dominates every condition block of the step-1 expression. Suppose it fails to dominate some condition block Y . Then Y is reached by a path avoiding B_p , so the producer's definition meets another value at a γ -gate at or before Y . There must be a predicate downstream of B_p which distinguishes the producer from the γ -gate's output, and this predicate is a vertex of the decision graph, contradicting that B_t is the nearest vertex.

If B_p is a γ -predicate, then B_t is the consumer block (shown above). Since B_t post-dominates B_p , no path from B_p bypasses the consumer, so a token is suppressed only through loop-back, and every condition block of the step-1 expression lies inside the loop. Suppose B_p did not dominate one such block. Then the block is reached both through B_p and through a path avoiding it; on the γ -tree, a predicate q must therefore decide whether the live in arrives through B_p . This q controls B_p 's predicate on the γ -tree, so it is a vertex of the decision graph, and it lies inside the same loop. Since every two conditional blocks of a loop are transitive control dependences of each other, B_p is a transitive control dependence of q and is therefore also a vertex of the decision graph, contradicting $B_p \notin C$. Hence B_p dominates every condition block.

In all cases B_p dominates the conditions of step 1, so the delivery (B_p, B_t) is a standard non- γ suppression and needs none of the dominator-block machinery of Section 5.7.

A token-semantics fact. We use the following observation, whose proof is immediate. If a block A dominates a block B , B post-dominates A , and the two lie in the same loops, then A and B emit tokens under the same conditions, so a delivery or suppression from one may reuse the logic for the other.

Reusing the demotion of B_t . Either B_p dominates B_t or $B_t = B_c$. Indeed, suppose B_p does not dominate B_t . Then B_t is reached both through B_p and through a path avoiding B_p , so the two incoming values meet at a γ -gate at or before B_t . Since B_t post-dominates B_p , the value produced at B_p is overwritten by the merged value before it can be live out at B_t . Thus the producer's value is not delivered beyond B_t and cannot appear on the γ -tree as the value coming from B_p . This is possible only when B_t is already the consumer block: in that case the value has been delivered at B_t itself, so no live-out beyond B_t is required. Hence $B_t = B_c$.

If $B_t = B_c$, no demotion is needed. After step 1, the surviving value has already been brought to the same level as the consumer. Since the consumer is a vertex of the decision graph at level 0, the producer's value does not need any further demotion. Algorithmically, step 2 is also skipped because the consumer has no demotion branches.

Otherwise B_p dominates B_t . Step 1 suppresses every token of B_p except the one that proceeds to B_t , lifting the surviving value out of the deeper loop to the level of B_t . This value and B_t then lie in the same loops. With B_p dominating B_t and B_t post-dominating B_p , the token-semantics fact gives that the value behaves like the condition token of B_t . It is therefore routed through the same demotion logic for B_t in Section 5.6, completing the construction.